

Оглавление

1	Основные составляющие РМД	2
1.1	Типы и домены.....	2
1.2	Структуры данных	3
1.3	Ключи в базе данных.....	5
2	Логическая схема реляционной базы данных (РБД)	6
2.1	Основные компоненты схемы и их получение	6
2.2	Таблица и ее поля	6
2.3	Связи и внешние ключи	7
2.3.1	Основные типы связей	7
2.3.2	Множественные ассоциации	8
3	Реализация отношений в РСУБД.....	10
3.1	Создание схемы данных.....	10
3.2	Вставка, удаление и обновление данных	12
4	Ограничения целостности и SQL	14
4.1	Реализация ограничений целостности	14
4.2	Время проверки ограничений в SQL	15
4.3	Способы задания ограничений целостности отношения на SQL	15

1 ОСНОВНЫЕ СОСТАВЛЯЮЩИЕ РМД

Отношение	целое	строка		целое		Типы данных
	номер	имя	должность	деньги		Домены
	Табельный номер	Имя	Должность	Оклад	Премия	Атрибуты
	2934 2935 2936	Иванов Петров Сидоров	инженер вед. инженер бухгалтер	112 144 92	40 50 35	Кортежи

↑
Ключ

1.1 Типы и домены

Понятие типа данных в реляционной модели данных полностью соответствует понятию типа данных в языках программирования. Состоит из трех основных компонентов:

- определение множества значений данного типа;
- определение набора операций, применимых к значениям типа;
- определение способа внешнего представления значений типа (литералов)

домен определяется путем задания:

- некоторого **базового типа данных**, к которому относятся элементы домена, и
- произвольного логического выражения, применяемого к элементу этого типа данных (*ограничения домена*).

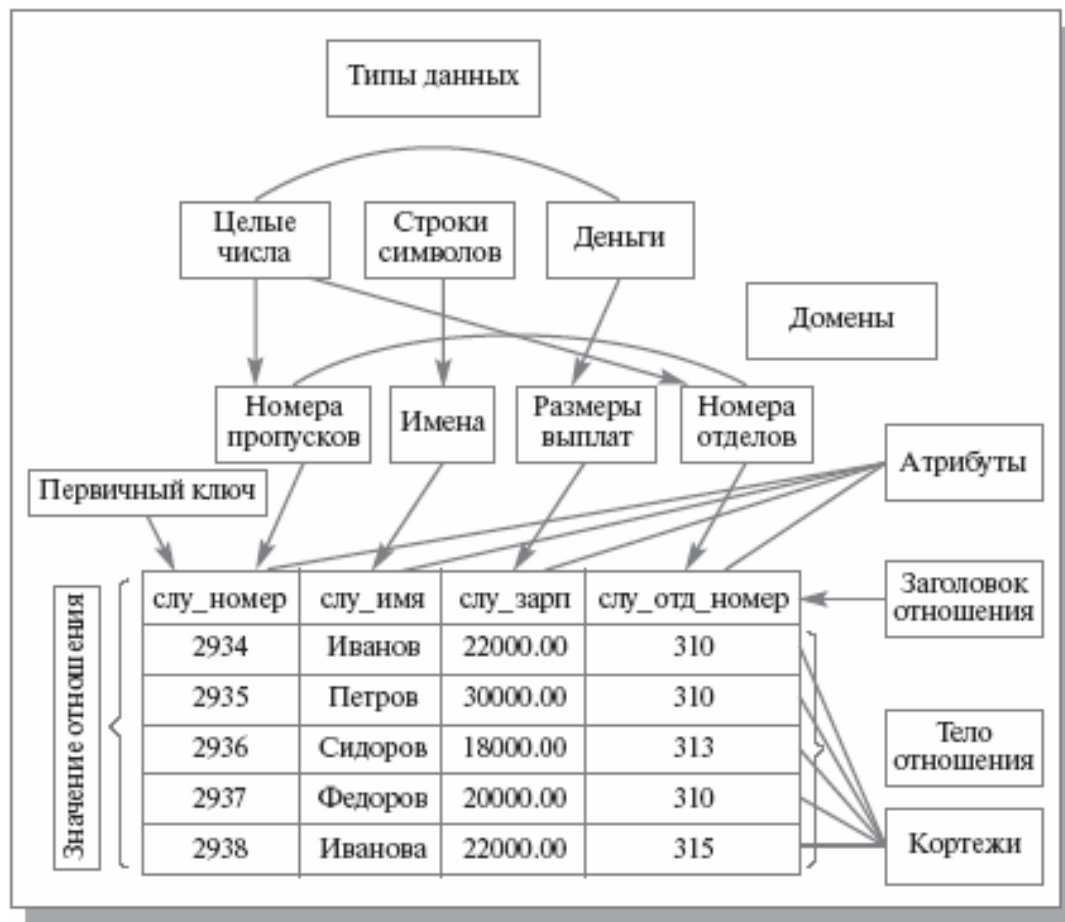
Элемент данных является элементом домена в том и только в том случае, если вычисление этого логического выражения дает результат истина.

С каждым доменом связывается **имя**, уникальное среди имен всех доменов соответствующей базы данных.

Домен - допустимое потенциальное, ограниченное подмножество значений данного типа

Данные считаются сравнимыми только в том случае, когда они относятся к одному домену

1.2 Структуры данных



n-арное отношение является единственной родовой структурой данных, хранящихся в реляционной базе данных.

Заголовком (или схемой) отношения r (H_r) называется конечное множество упорядоченных пар вида $\langle A, T \rangle$, где A называется именем атрибута, а T обозначает имя некоторого базового типа или ранее определенного домена. По определению требуется, чтобы все имена атрибутов в заголовке отношения были различны

Кортежем tr , соответствующим заголовку H_r , называется множество упорядоченных триплетов вида $\langle A, T, v \rangle$, по одному такому триплету для каждого атрибута в H_r . *Третий элемент – v – триплета $\langle A, T, v \rangle$ должен являться допустимым значением типа данных или домена T .*

Свойства кортежей:

1. Каждый кортеж содержит точно одно значение (соответствующего типа) для каждого из своих атрибутов.
2. Для компонентов кортежа не предусмотрено упорядочение. *Это свойство следует из того, что понятие кортежа определено на основании множества компонентов, а в математике множества не характеризуются упорядочением своих элементов.*
3. Каждое подмножество кортежа представляет собой кортеж (а каждое подмножество заголовка является заголовком). Более того, эти свойства являются истинными применительно, в частности, к пустым подмножествам. (см. следующий абзац).

Телом B_r отношения r называется произвольное множество кортежей tr .

Значением V_r отношения r называется пара множеств H_r и B_r .

В изменчивой реляционной базе данных хранятся отношения, значения которых изменяются во времени.

Переменной отношения VARr называется именованный контейнер, который может содержать любое допустимое значение Vr. Естественно, что при определении любой VARr требуется указывать соответствующий заголовок отношения Hr.

Здесь стоит подчеркнуть, что любая принятая на практике операция обновления базы данных:

INSERT (вставка кортежа в переменную отношения),

DELETE (удаление кортежа из значения-отношения переменной отношения) и

UPDATE (модификация кортежа значения-отношения переменной отношения) –

с модельной точки зрения является операцией присваивания переменной отношения некоторого нового значения-отношения. Это совсем не означает, что перечисленные операции должны выполняться именно таким образом в СУБД: главное, чтобы результат операций соответствовал этой модельной семантике.

По определению, **степенью, или «арностью»**, заголовка отношения, кортежа, соответствующего этому заголовку, тела отношения, значения отношения и переменной отношения является мощность заголовка отношения¹.

Схемой реляционной базы данных считается набор пар <имя VARr, Hr>, включающий имена и заголовки всех переменных отношения, которые определены в базе данных.

Реляционная база данных – это набор пар <VARr, Hr> (конечно, каждая переменная отношения в любой момент времени содержит некоторое значение-отношение, в частности, пустое).

Заметим, что в классических реляционных базах данных после определения схемы базы данных могли изменяться только значения переменных отношений. Однако теперь в большинстве реализаций допускается и изменение схемы базы данных: определение новых и изменение заголовков существующих переменных отношений. Это принято называть **эволюцией схемы базы данных**.

Первичным ключом переменной отношения является такое подмножество S множества атрибутов ее заголовка, что в любое время значение первичного ключа (*составное, если в состав первичного ключа входит более одного атрибута*) в любом кортеже тела отношения отличается от значения первичного ключа в любом другом кортеже тела этого отношения, а никакое собственное подмножество S этим свойством не обладает.

Обычным житейским представлением отношения является таблица, заголовком которой является схема отношения, а строками – кортежи отношения-экземпляра; в этом случае имена атрибутов соответствуют именам столбцов данной таблицы. Поэтому иногда говорят про «столбцы таблицы», имея в виду «атрибуты отношения».

Свойства отношений.

1. Отсутствие кортежей-дубликатов. Из этого свойства вытекает наличие у каждого кортежа первичного ключа. Для каждого отношения, по крайней мере, полный набор

¹ Напомним, что заголовок отношения это множество и в данном случае это – мощность множества заголовка

его атрибутов является первичным ключом. Однако, при определении первичного ключа должно соблюдаться требование "минимальности", т.е. в него не должны входить те атрибуты, которые можно отбросить без ущерба для основного свойства первичного ключа - однозначно определять кортеж.

2. Отсутствие упорядоченности кортежей (в пост реляционных СУБД кортежи неявно упорядочены, как и атрибуты).
3. Для ссылки на значение атрибута всегда используется имя атрибута. (Отсутствие упорядоченности атрибутов – см. кортежи)
4. Атомарность значений атрибутов, т.е. среди значений домена не могут содержаться множества значений (отношения) – см. кортежи

1.3 Ключи в базе данных

Ключ или потенциальный ключ – это минимальный набор атрибутов, по значениям которых можно однозначно выбрать требуемый экземпляр сущности. Минимальность означает, что исключение из набора любого атрибута не позволяет идентифицировать сущность по оставшимся. Каждая сущность должна но не обязана обладать хотя бы одним возможным ключом.

Ключи можно разделить по признаку общности как:

1. **Простой** – сформирован из значений единственного поля, которые однозначно определяют каждую запись
2. **Составной** – сформирован из значений нескольких полей, применяется при отсутствии простого ключа с точки зрения предметной области

По способу возникновения:

1. **Естественный** – основан на уже существующем поле. Например, поле фамилия.
 - a. + Меньший объем данных
 - b. + Более быстрый поиск по ключу
 - c. - Изменяемый
2. **Суррогатный** – основан на добавленном искусственным путем отдельном поле для однозначной идентификации.
 - a. + Неизменяемость
 - b. + Гарантированная уникальность
 - c. +- Эффективность при отсутствии удобных естественных ключей.
 - d. - Дополнительное поле => увеличение общего объема
3. **Интеллектуальный ключ** – основан на естественном ключе путем добавления дополнительного поля. Например:

Ключ	Фамилия	Имя	Отчество
ТИА	Тихонов	Игорь	Алексеевич

Применяется крайне редко и лишь для при необходимости добавления сокращенных кодов(напр. ТИА-0701)

По ограничению целостности данных:

· **Первичный ключ** – служит как ограничение целостности в рамках одной таблицы для однозначной идентификации, конкретно поле первичного ключа не может повторяться или быть пустым.

· **Внешний ключ(вторичный)** – служит как ограничение целостности связей нескольких таблиц, конкретно подчиненная таблица не может ссылаться на несуществующие записи главной таблицы(что позволяет строить целостные модели данных).

2 ЛОГИЧЕСКАЯ СХЕМА РЕЛЯЦИОННОЙ БАЗЫ ДАННЫХ (РБД)

2.1 Основные компоненты схемы и их получение

Логическая схема – это следующий шаг в процессе создания базы данных после построения концептуальной ER модели (схемы).

Она строится на основе концептуальной схемы, с учётом требований к системе, таких, как необходимая функциональность, производительность. По физической схеме строится сценарий генерации базы данных.

По внешнему виду физическая схема очень напоминает концептуальную, т.к. объекты концептуальной схемы в большинстве случаев заменяются объектами физической схемы.

Таблица соответствия понятий и объектов:

Концептуальная схема	Логическая схема
Сущность	Отношение - Таблица (table)
Экземпляр	Кортеж или Запись таблицы (record)
Атрибут	Атрибут, также поле записи, колонка (field, column)
Домен	Тип данных и ограничитель уровня поля (data type with it's constraint)
Ключ первичный	Ключ первичный (primary key); уникальность и обязательность подразумеваются
Ключ альтернативный	Уникальный индекс и требование обязательности (unique not null)
Связь – наследование, агрегирование, ссылка	Мигрирующие атрибуты и ограничитель ссылочной целостности (foreign key with referential integrity constraint)
Связь – множественная ассоциация	Дополнительная таблица с ссылками на ассоциированные таблицы

2.2 Таблица и ее поля

Справедливо будет сказать, что таблица – это самый важный объект базы данных, потому, что таблица хранит в себе данные. Данные хранятся в виде записей. Запись таблицы соответствует кортежу реляционной схемы и экземпляру сущности.

Следует обратить внимание, что существует заблуждение, якобы записи таблицы хранятся в порядке следования значения ключа; это не так – **записи хранятся как попало**; более того, части одной записи могут располагаться между частями других записей. Более подробно это будет рассмотрено в разделе по физическому хранению данных.

Количество записей в большинстве случаев бывает неограниченно какими-то специальными правилами, однако, оно **ограничено техническими характеристиками** используемого аппаратно-программного обеспечения.

Поле записи можно считать типизированной «переменной» базы данных; т.к. поле хранит значение некоторого типа. Понятие тип данных в реляционной модели данных полностью адекватно понятию типа данных в языках программирования.

Обычно в современных реляционных базах данных допускается хранение

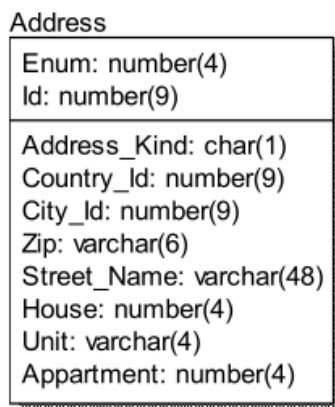
- символьных данных,
- числовых данных,
- битовых строк,
- специализированных числовых данных (таких как "деньги"), а также
- специальных данных (дата, время, временной интервал).

При создании таблицы следует описать все поля будущих кортежей, или иначе, колонки таблицы. Каждая колонка определяется именем и своими свойствами. Свойства колонок:

- тип данных,
- обязательность/опциональность,
- значение по умолчанию,
- ограничители уровня поля.

Каждое поле может иметь сколько угодно ограничителей.

В ER-диаграммах таблица обозначается так:



В верхней части прямоугольника перечисляются поля первичного ключа, ниже – остальные поля таблицы (иногда их просто подчеркивают)

2.3 Связи и внешние ключи

2.3.1 Основные типы связей

На логической схеме связи показаны в виде сплошных и пунктирных линий. Сплошная линия означает идентифицирующую связь, т.е. **атрибуты мигрируют в первичный ключ таблицы**; а пунктирные неидентифицирующую, атрибуты мигрируют в тело таблицы.

Чёрная точка указывает таблицу, в которую мигрируют атрибуты;

Прозрачный ромбик (только для неидентифицирующих связей) означает, что ссылка – опциональная (т.е. значения мигрировавших атрибутов могут быть не заданы).

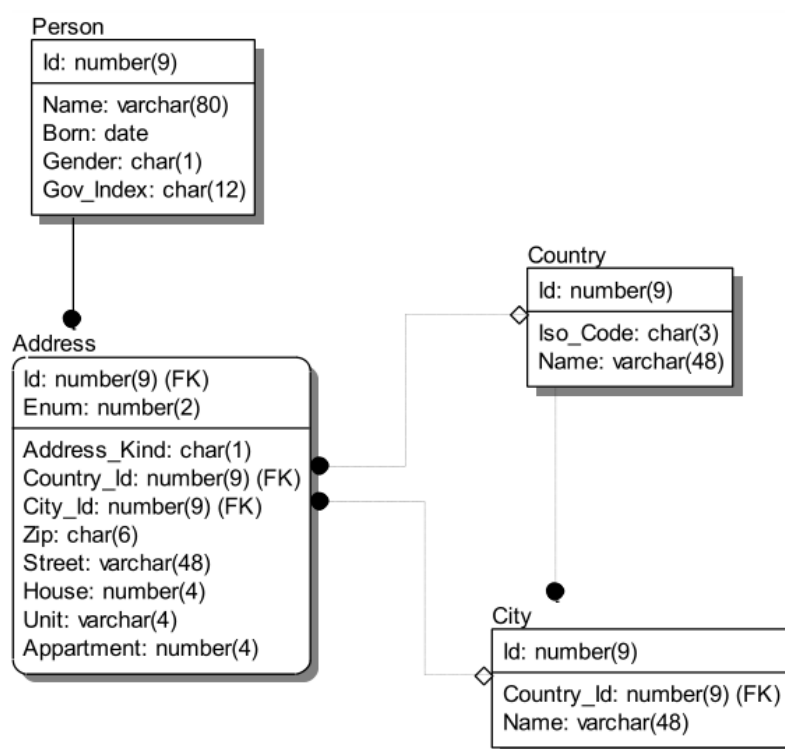
Мигрирующие атрибуты называют внешним ключом.

Таблица соответствия связей концептуальной схемы со связями логической схемы:

Концептуальная связь	Логическая связь
Иерархия/наследование	Идентифицирующая связь; причём первичный ключ полностью совпадает с внешним ключом
Агрегирование	Идентифицирующая связь; первичный ключ состоит из первичного внешнего ключа и ещё одного или нескольких атрибутов
Ссылка	Неидентифицирующая связь

Для связи можно указать опцию каскадного удаления/обновления и т.д.; обычно эту опцию указывают для иерархии/наследования и агрегирования.

Пример физической схемы:



Следует заметить, что порядок следования полей в первичном ключе очень важен; этому порядку должен соответствовать порядок следования мигрирующих полей.

2.3.2 Множественные ассоциации

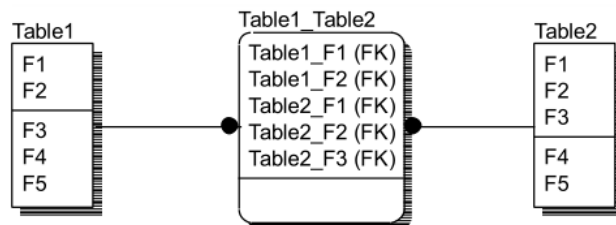
Множественная ассоциация реализуется с помощью **специальной дополнительной таблицы**, в которую мигрируют ключи из ассоциированных таблиц.

Все поля дополнительной таблицы являются её первичным ключом.

Например, концептуальная схема



превратится в физическую схему



3 РЕАЛИЗАЦИЯ ОТНОШЕНИЙ В РСУБД

3.1 Создание схемы данных

В реляционном сервере схема базы данных реализуется в виде скриптов на языке SQL. При этом лучше не пользоваться визуальными средствами проектирования с автоматической генерацией кода, так как в этом случае будет сложнее задать все необходимые характеристики схемы данных и обеспечить ее соответствие концептуальной модели.

Единица языка SQL - оператор (statement), состоящий из нескольких предложений (clause). Для создания, изменения и удаления таблиц используется часть SQL, называемая - язык определения данных (DDL, Data Definition Language).

Пользовательские базы данных как правило содержат два типа объектов:

1. Пользовательские объекты (таблицы, пользовательские типы данных, триггеры, хранимые процедуры, ограничения, индексы);
2. Системные объекты (таблицы, представления, хранимые процедуры, функции, типы данных, пользователи, роли, схемы).

Создать новую базу данных также можно при помощи SQL – скрипта, пример которого приведен ниже (здесь и далее примеры приведены для диалекта T-SQL).

Пример 1

```
USE master;
CREATE DATABASE DB_Test1
ON (NAME= DB_Test1,
    FILENAME = 'C:\DBTest_1.mdf',
    SIZE = 10,
    MAXSIZE = 100,
    FILEGROWTH = 5)
LOG ON
(NAME= DB_Test1,
    FILENAME = 'C:\DBTest_1.ldf',
    SIZE = 40,
    MAXSIZE = 100,
    FILEGROWTH = 10);
GO
```

Также можно применить изменение логического имени базы данных (пример 2) или изменить порядок сортировки базы данных (пример 3).

Пример 2

```
USE master;
ALTER DATABASE DBTest_1
MODIFY NAME = NewDBTest;
GO
```

Пример 3

```
USE master;
ALTER DATABASE DBTest_1
COLLATE Cyrillic_General_CI_AS;
GO
```

Основными параметрами базы данных, на которые стоит обратить внимание, являются следующие:

AUTO_CREATE_STATISTICS { ON | OFF } — задает или отключает автоматическое создание статистики по базе данных, требуемой для оптимизации запроса.

ANSI_NULL_DEFAULT { ON | OFF } — определяет значение по умолчанию для столбцов таблиц: NULL или NOT NULL.

ANSI_NULLS { ON | OFF } — задает соответствие стандарту ANSI результат сравнения любых значений со значением NULL.

ANSI_WARNINGS { ON | OFF } — влияет на появление предупреждающих сообщений или сообщений об ошибке при появлении значения NULL в агрегатных функциях или при делении на ноль.

ARITHABORT { ON | OFF } — определяет реакцию системы на арифметическое переполнение или при делении на ноль: прекращение выполнения запроса или выдача сообщения и продолжение выполнения запроса.

COMPATIBILITY_LEVEL = { 80 | 90 | 100 | 110 } — уровень совместимости с предыдущими версиями SQL Server.

CONCAT_NULL_YIELDS_NULL { ON | OFF } — определяет результат конкатенации двух строковых данных, когда одно из строковых значений имеет значение NULL.

Пример создания двух таблиц: должности и работника связанных ссылкой приведен ниже в примере.4.

Пример 4

```
use DBTest_1;
go

Create table TDoljnost(
    Doljnost nvarchar(30) not null,
    Oklad money,
    Salary as (Oklad *0.87),
    Constraint PK_TDoljnost primary key (Doljnost),
);
Go

Create table TEmployers(
    Employer nvarchar(30) NOT Null ,
    SocialNumber int not null,
    fm nvarchar(1) NOT NULL,
    Doljnost nvarchar(30),
```

```

Constraint PK_TEmployer primary key (SocialNumber),
constraint FK_TDoljnost foreign key (Doljnost) references
TDoljnost (Doljnost)
on update cascade on delete no action
);
Go

```

При реализации схемы данных важен порядок создания таблиц. В частности, когда создается ссылка на таблицу (например, ссылка на таблицу TDoljnost в таблице TEmployer) таблица, на которую ссылаются, должна уже существовать.

Если необходимо создать циклические связи, то можно воспользоваться командой изменения таблицы с последующим добавлением ограничения (alter table... add constraint).

Обратите внимание на вычисляемое поле Salary в таблице TDoljnost.

Также для удобства рефакторинга базы данных в скрипте создания можно предусмотреть возможность существования таблицы и, если такая таблица уже существует, заново пересоздать ее. Соответствующий код приведен в примере 5.

Пример 5

```

if exists (
    select *
    from sys.tables
    where name='TAddress'
)
drop table TAddress;
go

```

В данном примере проверяется существование таблицы TAddress в базе данных и, если такая таблица существует, она удаляется. Выражение if exists имеет смысл ставить перед скриптом создания каждой таблицы.

Для создания суррогатных ключей и генерации абстрактных значений в СУБД используются генераторы последовательностей. Это объекты уровня базы данных синтаксис и специфика работы которых зависят от конкретной СУБД.

3.2 Вставка, удаление и обновление данных

Вставка, удаление и обновление данных производится специальными SQL – операторами. Стандарт языка включает три основные операции добавления, удаления и изменения данных. Это INSERT, DELETE и UPDATE соответственно. Использование инструкции INSERT приведено в примере 6.

Пример 6.

```

insert into TAddress (Country, City, Street, House, Room)
values
( 'Россия', 'Санкт-Петербург', 'Московская', '23', 11),
( 'Германия', 'Мюнхен', 'Восточная', '12', null),

```

(N'Германия', N'Берлин', N'Карла Маркса', N'56', 3),
(N'Россия', N'Санкт-Петербург', N'Ленсовета', N'45а', 8),
(N'Белоруссия', N'Минск', N'Гагарина', N'29', null),
(N'Россия', N'Санкт-Петербург', N'Туполева', N'11', 4);

Использование «N» перед текстовой строкой обуславливает ее корректное отображение в кириллической кодировке.

Изменение данных может проводиться как на некотором диапазоне, так и на конкретном кортеже, выбранном по условию (пример 7).

Пример 7.

Update TEmployers

set Doljnost=N'Специалист'

where Employer=N'Иванов';

В данном примере у работника «Иванов» изменяется значение должности на «Специалист». Если в результате выполнения части WHERE будет выбран диапазон кортежей, изменения будут внесены на всем диапазоне.

В качестве устанавливаемого нового значения для столбца могут использоваться: константа (как в примере), выражение, ключевое слово DEFAULT (устанавливающее значение по умолчанию) или NULL.

Инструкция для удаления данных также достаточно проста и представлена в примере 8.

Пример 8.

delete from TAddress

where City != N'Санкт-Петербург';

Приведенная инструкция удаляет из таблицы TAddress все строки, в которых значение поля City не равно «Санкт-Петербург».

4 ОГРАНИЧЕНИЯ ЦЕЛОСТНОСТИ И SQL

4.1 Реализация ограничений целостности

В языке SQL вообще не поддерживаются ограничения типа, за исключением тех примитивных ограничений, которые являются прямым следствием применения определенного физического представления.

Пример:

Допустимо утверждать, что значения типа WEIGHT должны быть представлены в виде чисел DECIMAL (5,1), но нельзя указать, что эти числа должны быть больше нуля и меньше 5000

Вариант: пользовательские типы данных.

В языке SQL (безусловно) поддерживаются ограничения атрибута.

В языке SQL не поддерживаются ограничения переменной отношения как таковые. В нем обеспечивается поддержка ограничений базовой таблицы, но такие ограничения распространяются именно только на базовые таблицы, а не на все таблицы в целом, и они не сводятся к упоминанию просто одной такой таблицы, но фактически могут иметь произвольную сложность.

В языке SQL не поддерживаются ограничения базы данных как таковые. В нем поддерживаются общие ограничения, которые в спецификации этого языка называются **утверждениями (assertion)**, но не обязательно нужно использовать лишь такие ограничения, если требуется указать в ограничении больше одной таблицы.

```
CREATE ASSERTION constraint_name
```

```
CHECK (conditional_expression)
```

К работе на предприятии допускаются только те лица, которые родились после Октябрьского переворота. В логическом условии этого общего ограничения выбирается минимальное значение столбца EMP_BDATE (дата рождения самого старого служащего). Значением условного выражения будет false в том и только в том случае, если среди служащих имеется хотя бы один, родившийся до указанной даты.

```
CREATE ASSERTION MIN_EMP_BDATE CHECK
```

```
((SELECT MIN(EMP_BDATE)) FROM EMP) >= '1917-10-24')
```

Или для таблицы служащих

```
CHECK (EMP_BDATE >= '1917-10-24')
```

Удаление ограничения:

```
DROP ASSERTION <constraint name> ;
```

В языке SQL отсутствует также непосредственная поддержка **ограничений перехода**, но такие ограничения могут быть реализованы процедурно с помощью триггеров.

Пример:

```
Create table TEmployers(  
  Employer nvarchar(30) NOT Null ,  
  SocialNumber int not null,  
  fm nvarchar(1) NOT NULL,  
  Doljnost nvarchar(30),
```

```

Constraint PK_TEmployer primary key (SocialNumber),
constraint Check_fm check (fm in (N'м',N'ж')),

constraint FK_TDoljnost foreign key (Doljnost) references
    TDoljnost (Doljnost)
    on update cascade on delete no action
);

```

4.2 Время проверки ограничений в SQL

В языке SQL ограничения могут быть определены как допускающие отложенную проверку (DEFERRABLE) или не допускающие такую проверку (NOT DEFERRABLE);

Если заданное ограничение объявлено как DEFERRABLE, оно может быть дополнительно определено как отложенное первоначально (INITIALLY DEFERRED) или немедленно выполняемое с самого начала (INITIALLY IMMEDIATE); эти ключевые слова определяют **состояние ограничения в начале каждой транзакции**.

Ограничения NOT DEFERRABLE всегда проверяются немедленно, а проверку ограничений DEFERRABLE можно динамически включать и выключать с помощью следующего оператора.

```
SET CONSTRAINTS <constraint name commalist> <option> ;
```

Здесь опция <option> принимает значение IMMEDIATE или DEFERRED. Пример:

```
SET CONSTRAINTS SSP5, SSP6 DEFERRED ;
```

Ограничения DEFERRABLE проверяются, только если они находятся в состоянии IMMEDIATE. Перевод ограничения DEFERRABLE в состояние IMMEDIATE вызывает немедленную проверку этого ограничения; если данная проверка оканчивается неудачей, то и выполнение оператора SET IMMEDIATE оканчивается неудачей.

Выполнение оператора COMMIT влечет за собой вызов операторов SET IMMEDIATE для всех ограничений DEFERRABLE; если после этого хотя бы одна из проверок целостности оканчивается неудачей, происходит откат транзакции.

4.3 Способы задания ограничений целостности отношения на SQL

Декларативные:

- Уникальность
- Первичный ключ
- Внешний ключ
- Проверочное ограничение (Check)
- Значение по умолчанию (в т.ч. Null / Not Null)

Не декларативные:

- Триггеры

Декларативные ограничения целостности могут задаваться при создании или изменении таблиц базы данных или специальными операторами, определенными в языке SQL. К инструментам задания декларативных ограничений относятся:

Ключи отношений базы данных (первичные ключи; внешние ключи; атрибуты с уникальными значениями, представляющие альтернативные ключи отношений). Проверочные ограничения уровня атрибутов и отношений задаются оператором CHECK (пример 4 в предыдущей лабораторной работе).

Ограничение CHECK не поддерживает вложенных запросов и не может, таким образом, охватывать несколько отношений. Ограничения уровня базы данных, которые нельзя выразить этим оператором, называются *не декларативными* и задаются при помощи триггеров.

Триггеры не входят в программу данного курса и могут быть изучены студентом самостоятельно (при желании).